

SIT 218/738: Secure coding

Credit task 4.2C: Injection attack preventions using hibernate validator (part 2)

Objective

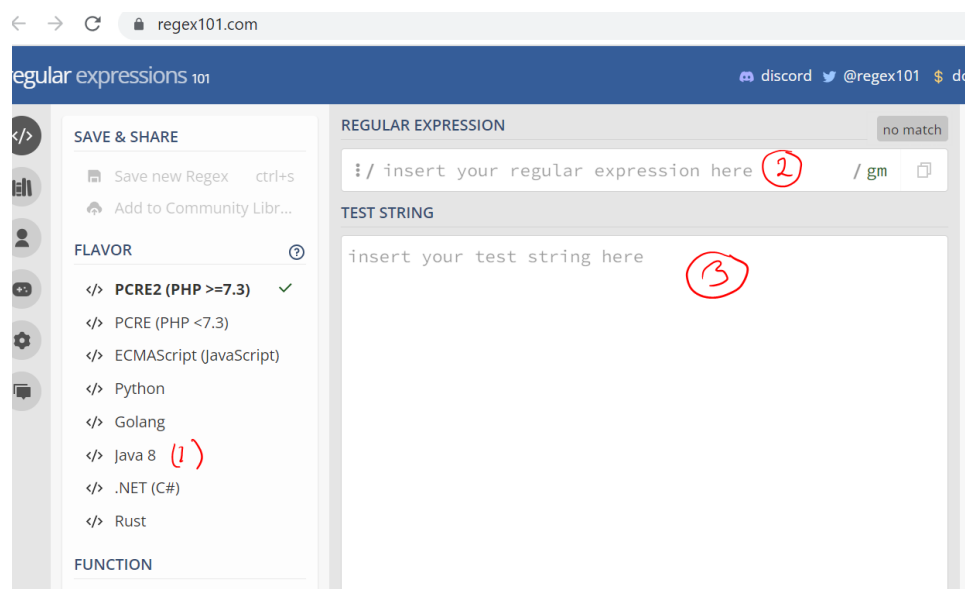
A poor regex for input validation can be easily bypassed and is also susceptible to resource exhaustion attacks. In this task you will build correct regex expression to reject script input and will focus on generating whitelist regex expression that can allow acceptable values and rejects all other values.

Overview

In 05-coachwebapp-spring web app we implemented the regex pattern to prevent users from entering the scripts or tagged input values in the format '<>'. This prevented many of the XSS attack input patterns. These filters can be bypassed using more clever inputs that do not get caught in the filters like the examples given below.

- 1) name "<script>alert('XSS'); </script>"
- 2) <a aa aaa aaaa aaaaa aaaaaa aaaaaaa aaaaaaaaa aaaaaaaaaa aaaaaaaaaa href=javascript:alert(1)>ClickMe
- 3) <script x> alert(1) </script 1=2

The regex expression does not prevent the above listed inputs. Correct the regex expression that can block these patterns as well but allows normal input to the name field like 'bob' to pass through. Navigate to the site <https://regex101.com/>. Use the screenshot below, follow the steps in the image below. Select the Flavor as Java 8 (stpe-1), enter the regex pattern given in (stpe-2) and enter the patterns you want to detect in (stpe-3) as shown below:

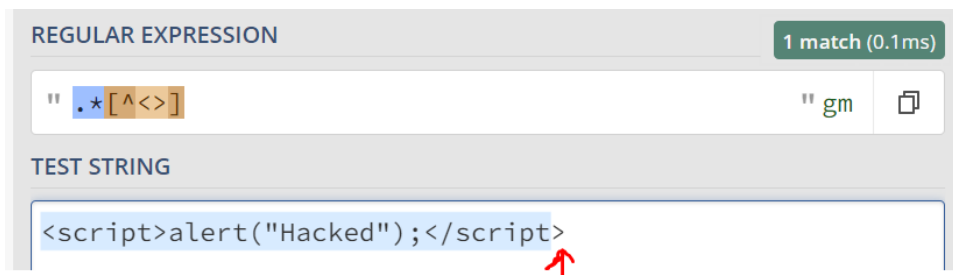


Example Activity:

Step-1: Select Java 8

Step-2: Enter the Regular Expression as `.*[<>]`

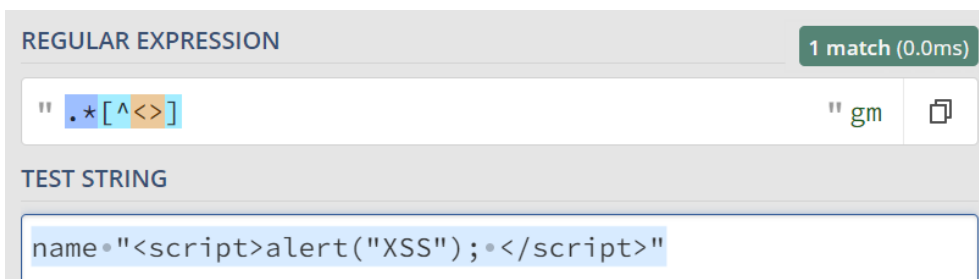
Step-3: Enter the test string as `<script>alert("Hacked");</script>`



The part of the inserted string that is highlighted in blue color is the match and will pass the regex filter and the un-highlighted part of the string will get rejected. So, in the first example, if the user input has the pattern ending with '`>`' it will get blocked.

Now enter the test string as: **name** `<script>alert("XSS");</script>`

By changing the input slightly our regex does not block the script input as you can see the test string is completely highlighted in blue and will get bypassed.



Task1: Now change the regular expression to below regex patterns:

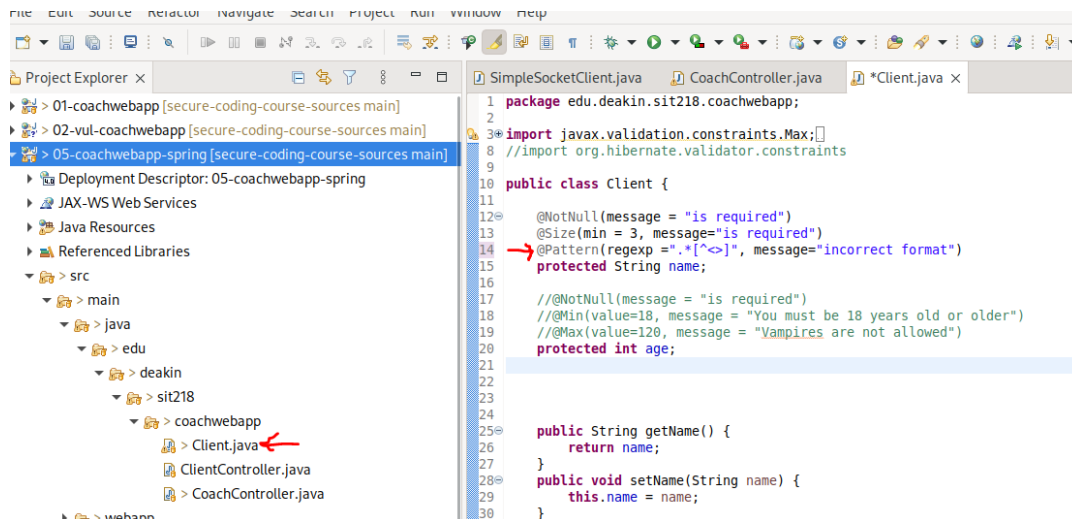
- `.*[<>]`
- `[<>].*`
- `[<>]+`
- `[a-zA-Z]+`

Among the given patterns which pattern(s) will block the characters in pattern like **name** `<script>alert("XSS");</script>`

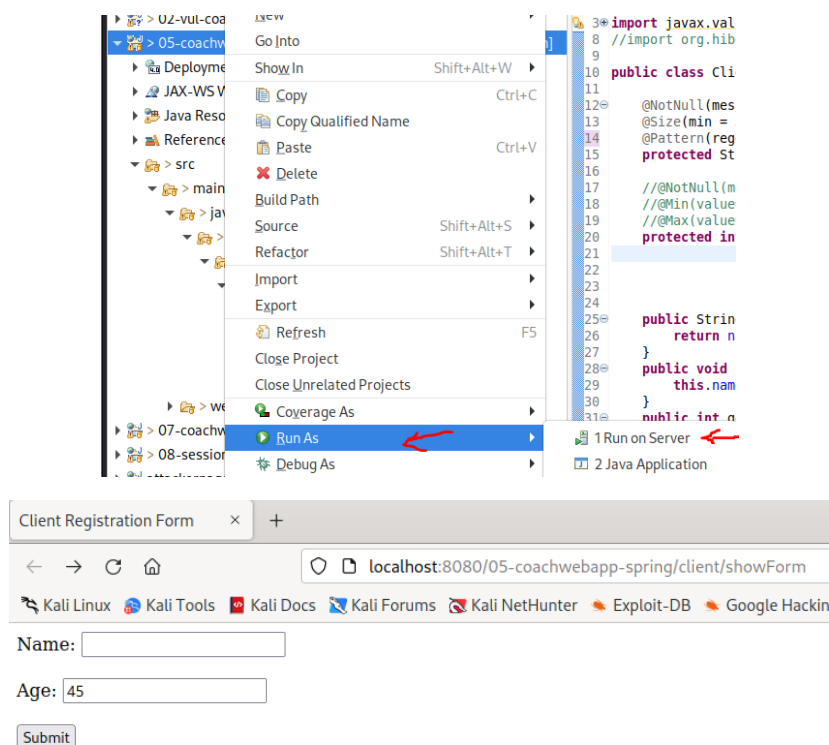
Task2: Whitelisting is preferred approach to only permit allowed characters and block everything else by default (read more on this approach here <https://owasp.deteact.com/cheat/cheatsheets/Input Validation Cheat Sheet.html>).

From the given regex patterns in the above list in Task-1, identify the whitelist pattern.

Task3: Once you selected the correct whitelist pattern in Task-2, change the regex pattern in the 05-coachwebapp-spring web app in the Eclipse IDE running on the provided KaliVM (see the figure below)



Save the changes and run the web application on the server and highlighted below:



In the webapp enter the name as "bob" and age 45 and confirm if this is not blocked. Next enter the input as **name "<script>alert('XSS'); </script>"** and test if it is getting blocked. Record screenshots add to the submission.

Submission Requirements:

Submit one PDF file containing the following information:

1. Screenshots of completion of step-1 to 3 of **Example Activity** section:
2. The correct regex pattern(s) that blocks the injected data from Task-1.
3. Whitelisting regex pattern from Task 2.

4. Screenshots showing that required regex pattern is working correctly as specified in Task3.